ChatGPT

Workplan: Building an iOS Proof-of-Concept for the Guided Storytelling Analytics App

This workplan accompanies `analysis_menu.md` and provides detailed tasks for implementing the prototype in Swift/SwiftUI. The goal is to build a simple iOS app (iPad-friendly) that allows parents to log a child's daily activities, view aggregated behaviour patterns, and select an analysis engine. The app will use SwiftData for storage, Charts for visualisation, and Core ML to optionally load a predictive model.

> **Note:** This is a proof of concept. The data model and analysis logic are intentionally simple and will need to be expanded in a production app. See `analysis_menu.md` for more context on the analytics design.

---

1 Project Setup

1. **Create a new Xcode project.** Choose *App* template, use SwiftUI and Swift Data (iOS 17+). Name it e.g. `StoryAnalyticsDemo`.
2. **Target iOS 17.0 or later.** This enables use of SwiftData and Charts.
3. **Enable Swift Data.** Add the `.modelContainer(for:)` modifier on the root view to configure the default model container. For example:

```
@main
struct StoryAnalyticsDemoApp: App {
    var body: some Scene {
        WindowGroup {
            ContentView()
        }
        .modelContainer(for: StoryEvent.self)
    }
}
```

---

2 Data Model and Dummy Data

1. **Define the `StoryEvent` model** in a new file. Annotate with `@Model` and create the properties as described in `analysis_menu.md`:

```
import SwiftData

@Model
final class StoryEvent {
    @Attribute(.unique) var id: UUID = UUID()
    var childName: String
```

```
    var childAge: Int
    var childGender: String
    var scene: Scene
    var place: Place
    var activity: String
    var mood: Mood
    var reason: String?
    var parentNote: String?
    var date: Date

    init(childName: String, childAge: Int, childGender: String,
         scene: Scene, place: Place, activity: String,
         mood: Mood, reason: String? = nil,
         parentNote: String? = nil, date: Date = Date()) {
        self.childName = childName
        self.childAge = childAge
        self.childGender = childGender
        self.scene = scene
        self.place = place
        self.activity = activity
        self.mood = mood
        self.reason = reason
        self.parentNote = parentNote
        self.date = date
    }
}
```

1. **Define enums** for `Scene`, `Mood`, and `Place` in the same or separate file. Conform them to `String`, `Codable`, and `CaseIterable` so they can be stored and used in UI pickers.

```swift enum Scene: String, Codable, CaseIterable, Identifiable { case morning, afternoon, evening, night var id: String { rawValue } }

enum Mood: String, Codable, CaseIterable, Identifiable { case happy, sad, angry, tired, excited, scared var id: String { rawValue } }

enum Place: String, Codable, CaseIterable, Identifiable { case home, school, park, other var id: String { rawValue } } ```

1. **Insert dummy data** for at least seven days. Create a helper function in a `PreviewProvider` or `onAppear` of `ContentView` that checks if there are no existing events and inserts sample events. Use random combinations of scene/place/mood. For example:

```
@MainActor
func preloadData(_ modelContext: ModelContext) {
    let existing = try? modelContext.fetch(FetchDescriptor<StoryEvent>())
    guard existing?.isEmpty ?? true else { return }
    let dateRange = (0..<7).map { Calendar.current.date(byAdding: .day,
value: -$0, to: Date())! }
    for day in dateRange {
```
```

```
        let event = StoryEvent(
            childName: "Leo",
            childAge: 4,
            childGender: "M",
            scene: .morning,
            place: .home,
            activity: "Breakfast",
            mood: .happy,
            reason: "Slept well",
            parentNote: "Woke up cheerfully",
            date: day
        )
        modelContext.insert(event)
    }
}
```

1. **Call** `preloadData` in `ContentView` on first launch: use `@Environment(\.modelContext)` to get the context.

---

3 Views

3.1 `ContentView` – Tab Bar

- Use a `TabView` with three tabs:
- **Data** – list of stored events.
- **Dashboard** – charts and summary.
- **Models** – select analysis engine.

Example skeleton:

```
struct ContentView: View {
    var body: some View {
        TabView {
            DataListView()
                .tabItem { Label("Data", systemImage: "tray.full") }
            DashboardView()
                .tabItem { Label("Dashboard", systemImage:
"chart.bar.xaxis") }
            ModelSelectionView()
                .tabItem { Label("Models", systemImage: "gearshape") }
        }
    }
}
```

3.2 `DataListView` – Viewing and Adding Data

- Fetch all `StoryEvent` entries with `@Query` sorted by date descending.

- Display them in a `List` with the date, scene, and mood. Use a `NavigationStack` to push a detail view if needed.
- Provide a `ToolbarItem` with an **Add** button to create a new dummy entry (for test purposes). This can call a function to insert a new `StoryEvent` into the context.
- Show an empty state message if there is no data.

3.3 `DashboardView` – Charts and Insights

- Use `@Query` to fetch events. Group events by properties to create datasets for charts.
- Build multiple charts using the **Charts** framework:
- **Mood by Scene** – a `BarMark` grouping by `scene` and using `.foregroundStyle(by: .value("Mood", event.mood.rawValue))`.
- **Mood by Place** – similar but group by `place`.
- **Mood over Time** – use `LineMark` with `x` as date and `y` as numeric values (e.g., assign a value to each mood or use a count per day).
- Display charts in a `ScrollView` or `List` for vertical stacking. Use `Card`-style containers or `Section` headers to separate them.
- Under the charts, compute a simple **insight summary** string. For example: calculate the most frequent mood and the scene with the highest count of negative moods. Present the summary in a `Text` view with some styling.

3.4 `ModelSelectionView` – Selecting Analysis Engine

- Provide a `List` or `Form` with sections:
- **Current Engine** – show the currently selected mode (e.g. “Simple Rules”). Store this selection in `@AppStorage` for persistence.
- **Available Models** – present options such as **Simple Rules** (default), **Core ML Classifier**, **LLM Summariser**.
- **Download Model** – if the user chooses a Core ML model or LLM that is not yet present, show a `Button` to download. Provide an input field for a model URL or use a predefined list of demo models.

- **Model Info** – when a model is selected, display its description (e.g. training data source, version). Use static strings for now.

- Implement model download logic using `URLSession.downloadTask`. Save the downloaded `.mlmodel` in the app’s documents directory. After downloading, compile the model with `MLModel.compileModel(at:)` and load it with `MLModel(contentsOf:)`.

- Provide a function to perform analysis using the selected model, for example classify mood given scene/place/activity. In this prototype, you can stub this functionality.

---

4 Core ML Integration (Optional)

1. **Train a simple model outside the app** (e.g. in Python using scikit-learn) to classify mood based on scene, place, and activity. Convert the model to Core ML format using `coremltools`. Store the `.mlmodel` file in the app bundle or download it at runtime.

2. **Load the model** using `MLModel`:

```
let modelURL = Bundle.main.url(forResource: "MoodClassifier", withExtension:
"mlmodelc")!
let mlModel = try! MLModel(contentsOf: modelURL)
let classifier = try! VNCoreMLModel(for: mlModel)
```

1. **Perform inference** in the dashboard to predict mood or cluster patterns. Use `MLDictionaryFeatureProvider` to convert feature data.

2. **Update the insight summary** with the predictions.

---

5 Additional Considerations

- **Accessibility:** Ensure voiceover labels are set on buttons and charts. Use `accessibilityLabel` on icon buttons.
- **iPad Layout:** Adopt `NavigationSplitView` or ensure the tab bar layout adjusts to iPad size. Use `Grid` or `LazyVGrid` to arrange charts elegantly on larger screens.
- **Persistence Testing:** Use Unit Tests to verify that data insertion and fetching via SwiftData works correctly.
- **Localization:** Externalize strings for mood and scene names. Use `LocalizedStringKey` in SwiftUI.
- **Ethical Use:** Remind users that the app offers reflective insights, not psychological diagnosis. Provide links to professional resources if parents have concerns about atypical behaviours.

Deliverables for Codex

1. **Swift Files**

2. `StoryEvent.swift` – model definition and enums.

3. `ContentView.swift` – tab bar layout.
4. `DataListView.swift` – listing and adding data.
5. `DashboardView.swift` – charts and summary.
6. `ModelSelectionView.swift` – engine selection, download and loading.

7. `Helpers.swift` – dummy data loader, chart utilities, file download helper.

8. **Model Files (Optional)**

9. `MoodClassifier.mlmodel` – a simple trained model (if used).

10. **Dummy Data** – Preloaded in the app for a week.

11. **Unit Tests** – Basic tests for data insertion and retrieval.

12. **Documentation** – README summarizing how to run the app and limitations.

This workplan should guide the implementation of the proof-of-concept analytics dashboard and model selection features for your storytelling app.
